Manual técnico — Proyecto Red (G2), primera entrega

Versión: etiqueta primera-entrega del repositorio. El informe correspondiente está en entregables/primera_entrega/1_Informe_Primera_Entrega_G2.pdf.

Solución base: selección de nodos (Parte 2) y reparto de presupuesto (Parte 1). Las demostraciones (NP-dureza, eficiencia) están en el informe; este manual describe el código.

1. Algoritmos

1.1 Parte 2 — SELECCIONAR-NODOS (voraz aproximado) · `red/seleccion.py`

En k pasos agrega al equipo S el nodo de mayor ganancia marginal Δ(v | S). Empates: el menor índice. El voraz no conoce f: se la pide a un oráculo con la interfaz `estado_inicial`, `ganancia`, `agregar` y `valor`. Con cobertura (formato propio), Δ(v | S) = |C_v \ cub(S)|. Con influencia (formato de la profesora), Δ(v | S) = a_v·q_v + Σ_{w vecino de v} a_w·p_vw·q_w.

Aspecto	Detalle
Operación básica	una evaluación de Δ (llamada al oráculo)
Conteo	k·N − k(k−1)/2, exacto y para toda entrada; entre kN/2 y kN ⇒ **Θ(N·k)** en mejor, peor y promedio
Tiempo con cobertura	O(k·(N + M)), con M = Σ |C_v|. El N pesa porque cada paso recorre V; si ningún C_v es vacío, N ≤ M y queda O(k·M)
Tiempo con influencia	O(k·(N + A)), con A = número de aristas: cada llamada cuesta O(1 + grado(v)). Con grafo completo, $O(k \cdot N^2)$
Espacio adicional	O(k) más el estado del oráculo (O(|U|) con cobertura, O(N) con influencia) sin traza; la ejecución con traza guarda además los Δ de cada paso, Θ(N·k)
Precondición	El modelo exige 1 ≤ k < N, y lo comprueba la verificación de entrada. El algoritmo por sí solo funciona con 1 ≤ k ≤ |V|, con V ordenado y sin repetidos. La valida la ronda; el algoritmo no la comprueba, para seguir el pseudocódigo
Garantía	f(S) ≥ (1 − 1/e)·OPT (Nemhauser, Wolsey y Fisher, 1978). Las dos f son monótonas, submodulares y con f(∅) = 0

1.2 Parte 1 — REPARTIR-PRESUPUESTO (voraz óptimo con montículo) · `red/reparto.py`

Entrega hasta B unidades una por una; cada unidad va al nodo cuya siguiente unidad aporta más. Montículo de máximos simulado con `heapq` (de mínimos) guardando parejas (−Δ_i(x_i), i). Empates: el menor índice. Un nodo que llega a su tope c_i no vuelve al montículo; si todos llegan, el ciclo termina y el sobrante queda sin gastar.

Aspecto	Detalle
Construir el montículo	O(N) (`heapify`)
Ciclo	B vueltas, cada una con un `heappop` y un `heappush` de O(lg N). Con topes, mín(B, Σ c_i) vueltas, y quien llega a su tope no hace `heappush`
Tiempo	peor caso **O(N + B lg N)**, también con topes. Sin topes y con `heapq`, **Θ(N + B(1 + lg N))** para toda entrada: `heappop` siempre baja hasta una hoja, así que no hay un mejor caso más barato. El 1 cubre N = 1: lg 1 = 0, pero el ciclo igual da B vueltas
Espacio	Θ(N)
Precondiciones	B ≥ 0, S sin repetidos, c_i entero ≥ 0, u_i crecientes y cóncavas. Las valida la ronda; el algoritmo no las comprueba, para seguir el pseudocódigo
Optimalidad	por concavidad de las u_i (argumento de intercambio, entrega posterior)

Dentro de una ronda, N de la Parte 1 es k (los nodos elegidos).

2. Estructura del código

Archivo	Responsabilidad
`ejecutar.py`	Línea de comandos: casos → ronda → evidencias
`red/entrada.py`	Leer y validar la forma del JSON en los dos formatos; único archivo que los conoce; crea el oráculo que corresponde
`red/utilidades.py`	Familias u_i (log, raiz, tabla) con Δ en O(1)
`red/cobertura.py`	Oráculo f de cobertura (formato propio) y contadores de llamadas
`red/influencia.py`	Oráculo f de cobertura por influencia (formato de la profesora) y contadores de llamadas
`red/seleccion.py`	Parte 2
`red/reparto.py`	Parte 1
`red/verificacion.py`	Restricciones de entrada y salida
`red/traza.py`	Paso a paso en texto y JSON
`red/ronda.py`	Orquesta la ronda y mide tiempos
`red/generador.py`	Instancias sintéticas con semilla

Flujo de una ronda:

```
leer_instancia → verificar_entrada → seleccionar_nodos (Parte 2) → repartir_presupuesto (Parte 1 sobre S)
              → verificar_seleccion / verificar_reparto → traza y tiempos → archivos
```

Cada archivo tiene su ficha en `docs/modulos/` con su complejidad, cómo modificarlo y preguntas de sustentación.

3. Formatos

`red/entrada.py` reconoce dos formatos de entrada: si el objeto tiene `"parametros"`, es el de la profesora (3.1); si no, es el propio (3.1.1). Los dos producen la misma salida (3.2).

3.1 Entrada: formato de la profesora

```
{
  "parametros": {
    "numero_nodos": 2,
    "numero_rondas": 3,
    "cupo_participantes_por_ronda_k": 1,
    "presupuesto_ruido_por_ronda_B": 30,
    "presupuesto_costos_participacion_por_ronda": 24
  },
  "grafo": [{"u": "N01", "v": "N02", "peso_influencia": 0.25}],
  "nodos": [
    {"id": "N01", "coeficiente_utilidad": 8.5, "presupuesto_total_nodo": 18, "costo_participacion": 3},
    {"id": "N02", "coeficiente_utilidad": 10.2, "presupuesto_total_nodo": 18, "costo_participacion": 5}
  ]
}
```

Campo	Uso en la solución base
`numero_nodos`	se comprueba contra la lista de nodos
`cupo_participantes_por_ronda_k`	k
`presupuesto_ruido_por_ronda_B`	B
`grafo`	aristas {u, v} no dirigidas, con p_vw = `peso_influencia`; definen f. El peso debe estar en [0, 1]
`id`	texto. Con ceros a la izquierda, el orden de texto coincide con el numérico y define los desempates. Sin ceros, "N10" iría antes que "N2"
`coeficiente_utilidad`	a de u_i y a_w de f. Debe ser un número finito > 0
`numero_rondas`, `costo_participacion`, `presupuesto_costos_participacion_por_ronda`	de las variantes (b) y (a): **se leen y se reportan** en `informativo`, pero no se aplican
`presupuesto_total_nodo`	tope c_i de x_i ≤ c_i. Entero ≥ 0

Supuestos de modelado adoptados (decisiones 24 a 26 y 28, en docs/agente/DECISIONES.md, y se declaran en el informe):

- f(S) = Σ_w a_w · (1 − Π_{v∈S} (1 − p_vw)), con p_vw = 0 si no hay arista y p_ww = 1.
- u_i(x) = `coeficiente_utilidad` · ln(1 + x).
- La solución base corre una sola ronda.
- `presupuesto_total_nodo` es lo que le queda al nodo para todo el proyecto, y en una ronda lo puede gastar completo. En la variante (b) se descontará ronda a ronda.

Es error de forma (`[ERROR]`): falta un campo; `numero_nodos` no coincide con la lista; una arista nombra un nodo inexistente, es un lazo o está repetida ({u, v} y {v, u} son la misma); un valor no es numérico (incluye `true`, `NaN` e `Infinity`); un id no es texto; k, B, `numero_nodos` o `numero_rondas` no son enteros; un coeficiente no es > 0; `presupuesto_total_nodo` no es un entero ≥ 0. Un peso fuera de [0, 1] no es error de forma, sino una verificación de entrada que falla (`[FALLA]`), y el caso no corre.

3.1.1 Entrada: formato propio (batería propia, informe 6.5)

```
{
  "k": 2,
  "B": 4,
  "nodos": [
    {"id": 1, "cubre": ["a", "b", "c"], "utilidad": {"tipo": "log",  "a": 9.0}},
    {"id": 2, "cubre": ["c", "d"],      "utilidad": {"tipo": "raiz", "a": 7.0}},
    {"id": 3, "cubre": ["d", "e"],      "utilidad": {"tipo": "log",  "a": 6.0}}
  ]
}
```

Campo	Significado
`k`	nodos a seleccionar (1 ≤ k < N)
`B`	unidades de presupuesto (entero ≥ 0)
`id`	entero único; define el orden y los desempates
`tope`	opcional: tope c_i de x_i ≤ c_i, entero ≥ 0. Sin él, el nodo no tiene tope
`cubre`	elementos que aporta el nodo (cadenas o números); definen f como cobertura
`utilidad`	`{"tipo": "log", "a": a}` → a·ln(1 + x); `{"tipo": "raiz", "a": a}` → a·√x; `{"tipo": "tabla", "ganancias": [Δ(0), Δ(1), …]}` → ganancias explícitas por unidad (para ejemplos y contraejemplos). `a` debe ser un número finito con a > 0, y las ganancias, números finitos. Valores como `true`, `"3"`, `NaN` o `Infinity` son error de forma

El archivo debe estar en UTF-8, con o sin BOM. En una carpeta, la extensión `.json` se reconoce sin distinguir mayúsculas. El nombre del caso es el del archivo sin esa extensión.

3.2 Salida

Por cada caso se escribe `<salida>/<caso>/salida.json`, `traza.json` y `traza.txt`. Además, `<salida>/resumen.csv` tiene una fila por caso.

`salida.json`

Campo	Contenido
`caso`	nombre del archivo, sin `.json`
`entrada`	N, k, B
`seleccion`	`S` (en el orden en que se eligió), `valor` = f(S), `llamadas_ganancia`, `llamadas_valor`
`reparto`	`x` (id → unidades), `utilidad` = Σ u_i(x_i), `extracciones`, `inserciones`
`tiempos_segundos`	`repeticiones` (R), y para `seleccion` y `reparto`: `mediana`, `minimo` y `maximo` de las R repeticiones (solo el algoritmo)
`verificaciones`	listas `entrada`, `seleccion`, `reparto`, `ejecucion`, cada una con nombre, cumple y detalle
`todas_cumplen`	true si todas las verificaciones cumplen
`informativo`	solo en el formato de la profesora, y solo si el caso corrió. `variante_a`: `suma_costos_de_S`, `presupuesto_costos_participacion_por_ronda` y `nota`. `variante_b`: `numero_rondas` y `nota`. La nota dice "no se aplica en la solución base". **No es una verificación ni entra en `todas_cumplen`**

`valor` y `utilidad` son números reales con toda su precisión; en el formato propio, `valor` es un entero.

`traza.json` tiene `pasos_seleccion` (por paso: `paso`, `aportes` con el Δ de cada candidato, `elegido`, `ganancia_elegido`, `valor_despues`) y `unidades_reparto` (por unidad: `unidad`, `nodo`, `ganancia`).

`traza.txt` es el reporte legible, con estas secciones en orden: encabezado (caso, N, k, B); verificación de la entrada; Parte 2 (Δ de cada candidato, elegido y f(S) por paso, y el resultado con las llamadas a Δ); Parte 1 (nodo y Δ de cada unidad, y el reparto con la utilidad total); verificaciones de Parte 2, Parte 1 y Ejecución; `Informativo` (solo en el formato de la profesora); mediana de los tiempos y R; y una línea final `TODAS LAS VERIFICACIONES: CUMPLEN` o `HAY FALLAS`. Muestra los números con 4 decimales; los valores exactos están en `salida.json`.

Caso rechazado: `salida.json` trae solo `caso`, `entrada`, `verificaciones.entrada` y `todas_cumplen = false`; `traza.json` trae las listas vacías y `motivo_rechazo`; `traza.txt` trae las verificaciones y el motivo. Si el archivo tiene forma inválida, N, k y B quedan en `null`. Si no se pudo ni abrir, no se escriben evidencias.

`<salida>/resumen.csv`: columnas `caso, N, k, B, S, f(S), utilidad_reparto, tiempo_seleccion_s, tiempo_reparto_s, minimo_seleccion_s, minimo_reparto_s, repeticiones, todas_cumplen`. S va separado por espacios, la utilidad con 6 decimales y los tiempos con 9: `tiempo_*` son las medianas y `minimo_*` los mínimos de las R repeticiones. **Para la gráfica de T(n) se usa el mínimo** (decisión 30). Tiene una fila por caso de la **última corrida**, sin acumular. Un caso rechazado deja vacíos S, f(S), utilidad, tiempos, mínimos y repeticiones, con `todas_cumplen` en False. Si ya existía, se reemplaza y se avisa por consola. Dos archivos con el mismo nombre de caso (sin distinguir mayúsculas) son un error: corre el primero y el segundo no se ejecuta ni tiene fila.

3.3 Verificaciones

Parte	Comprobación
Entrada	1 ≤ k < N; B ≥ 0; ids únicos; cada u_i creciente y cóncava en {0, …, B}; solo con influencia, cada peso en [0, 1]
Parte 2	S ⊆ V; sin repetidos; |S| ≤ k; valor acumulado = f(S) recalculada, con tolerancia 1e-9; llamadas = k·N − k(k−1)/2
Parte 1	x_i entero en [0, B]; si hay topes, x_i ≤ c_i; Σ x_i = mín(B, Σ c_i); utilidad finita e igual a Σ u_i(x_i) recalculada; extracciones = Σ x_i e inserciones = Σ x_i − nodos en su tope
Ejecución	todas las repeticiones dan el mismo S y el mismo reparto; la ejecución con traza da lo mismo que la medida

La concavidad se revisa solo en las unidades que se pueden entregar: en log y raíz basta a > 0 (O(1)); en tabla, Δ(x) ≥ 0 si x < B y Δ(x + 1) ≤ Δ(x) si x + 1 < B (O(min(L, B))).

Si la entrada no cumple sus restricciones, los algoritmos no se ejecutan y la traza lo reporta con el motivo.

4. Ejecución paso a paso

1. Instalar Python 3.11+. La ejecución usa solo la biblioteca estándar de Python; `pytest` es solo para las pruebas (`python -m pip install -r requirements-dev.txt`). En Windows el comando puede ser `py` en lugar de `python`.
2. Abrir una terminal en la raíz del repositorio.
3. `python -m pytest` → todas las pruebas deben pasar.
4. `python ejecutar.py casos/profesora --salida resultados/profesora` (o cualquier archivo o carpeta de casos).
5. Revisar `resultados/profesora/resumen.csv` y la carpeta de cada caso.

5. Casos de prueba utilizados

Caso oficial: `casos/profesora/entrada_G2_red_15_nodos.json` (N = 15, k = 5, B = 30), con evidencias en `resultados/profesora/`. Resultado: S = N11, N14, N09, N15, N13; f(S) ≈ 145.5795; 65 llamadas a Δ; reparto 5, 7, 7, 6, 5; U ≈ 88.5640; 30 extracciones y 30 inserciones. Todas las verificaciones cumplen. Depende de los supuestos de la sección 3.1. El informe reporta estos mismos valores en su sección 6.9.

Batería propia: 14 casos válidos y 3 inválidos, descrita caso por caso (qué comprueba, esperado y obtenido) en `casos/CATALOGO.md`, con evidencias en `resultados/propios/` y `resultados/invalidos/`.

`pruebas/test_bateria.py` comprueba automáticamente que los válidos cumplen todas las verificaciones, que los inválidos se rechazan y que el caso oficial da esos valores.

6. Medición de tiempos

Se usa `time.perf_counter()` alrededor de cada algoritmo, en ejecuciones **sin traza**; no incluye lectura, escritura, traza ni verificaciones. La ejecución medida se repite R veces (21 por omisión, `--repeticiones`), cada una con un oráculo nuevo, y se reporta la **mediana** de cada parte; `salida.json` trae también el mínimo y el máximo. Se verifica que todas las repeticiones den el mismo S y el mismo reparto. Luego se ejecuta otra vez con traza para la evidencia y se comprueba que coincide. En casos muy pequeños los tiempos son de microsegundos y tienen ruido del sistema: la mediana no se deja arrastrar por una repetición lenta, pero no resiste una ráfaga que cubra más de la mitad de las repeticiones (por eso para T(n) se usa el mínimo, que trae `salida.json` y `resumen.csv`: el ruido del sistema solo suma tiempo, así que el mínimo es la medición menos contaminada; decisión 30). La comparación contra T(n) se hará en la entrega de experimentación con instancias grandes y repeticiones.

7. Limitaciones

- El archivo de la docente no dice cómo usar sus datos: f, u_i y el alcance de una ronda son supuestos de modelado (decisiones 24 a 26, en docs/agente/DECISIONES.md), no confirmados todavía con la docente. Otro formato solo requiere cambiar `red/entrada.py`.
- La solución base no aplica `numero_rondas`, `costo_participacion` ni `presupuesto_costos_participacion_por_ronda`.
- El tope se aplica en una sola ronda y no se descuenta: eso es de la variante (b). Si los topes de S suman menos que B, el sobrante queda sin gastar. La selección no mira los topes, así que puede elegir un nodo con tope 0.
- La concavidad se juzga hasta B, no hasta c_i: una tabla que se dañe entre c_i y B se rechaza, aunque esas unidades nunca se entreguen.
- Otra función submodular requiere solo otro oráculo con la misma interfaz y devolverlo en `Instancia.crear_oraculo`.
- Con influencia, f es real: un empate exacto en teoría podría romperse por el redondeo y no por el menor índice. En el caso oficial no hay empates (la menor brecha es 0.1037, medida con aritmética exacta).
- Con ids de texto sin ceros a la izquierda, manda el orden de texto ("N10" antes que "N2").
- La Parte 2 es aproximada: el voraz no siempre da el óptimo (ejemplo: `casos/propios/contraejemplo_seleccion.json` da 5 y el óptimo es 6).
- La Parte 1 solo es óptima si cada u_i es cóncava; si no lo es, la verificación de entrada lo reporta y la ronda no se ejecuta.

- El contraejemplo convexo de la Parte 1 solo se puede mostrar por llamada directa a `repartir_presupuesto`; por `ejecutar.py` esa entrada se rechaza sin correr los algoritmos.
- La verificación de entrada cuesta O(N + Σ min(L_i, B)) esperado, con L_i el largo de cada tabla, y O(N + A) con influencia; no afecta los tiempos reportados.
- Con coeficientes extremos (a cerca de 1e308) los valores se desbordan a infinito: el caso se reporta como FALLA en la verificación de la utilidad, y `salida.json` puede traer `Infinity`, que no es JSON estándar.
- La ejecución con traza guarda Θ(N·k + B) anotaciones; con instancias muy grandes, la traza y sus archivos pesan.
- No se incluyen todavía: el algoritmo exacto de la Parte 2, las variantes de la sección 4.4 del enunciado ni la validación experimental.